📘 Class 7 — Scalable & Maintainable SCSS in Angular

- ☑ ~~View Encapsulation~~
- ☑ ~~:host~~
- ☑ ~~:host-context~~
- ☑ ~~Maintainable SCSS~~

1. View Encapsulation

Angular isolates component styles so they do not accidentally affect other components. This is called **View Encapsulation**. You set it in the `@Component` decorator:

```
import { Component, ViewEncapsulation } from '@angular/cor
e';

@Component({
  selector: 'app-example',
  templateUrl: './example.html',
  styleUrl: './example.scss',
  encapsulation: ViewEncapsulation.Emulated   // ← set mode
here
})
export class ExampleComponent {}
```

There are **4 modes**:

Mode 1 — `ViewEncapsulation.Emulated` (Default)

Angular **emulates** Shadow DOM by adding a unique generated attribute to every element in the component's template.

How it works:

```
<!-- Angular adds a unique attribute like _ngcontent-abc-c1
23 -->
<p _ngcontent-abc-c123>Hello</p>
```

```
/* Your CSS gets transformed to: */
p[_ngcontent-abc-c123] { color: red; }
```

Result:
- Component styles only apply to that component's own elements ✓
- Global styles (from `styles.scss`) still flow into the component ✓
- Component styles do NOT leak out to other components ✓

This is the default and the most commonly used mode.

Mode 2 — `ViewEncapsulation.ShadowDom`

Uses the **browser's native Shadow DOM API**. Angular attaches a real ShadowRoot to the component's host element.

How it works:

```
encapsulation: ViewEncapsulation.ShadowDom
```

Result:
- Component styles are truly isolated — they cannot affect anything outside ✓
- Global styles from `styles.scss` **cannot** penetrate into the component ✗

Mode 3 — `ViewEncapsulation.ExperimentalIsolatedShadowDom`

Similar to `ShadowDom` but with **stricter isolation guarantees**.

```
encapsulation: ViewEncapsulation.ExperimentalIsolatedShadow
Dom
```

> **Note:** This is marked **experimental** — its behavior may change in future Angular versions. Do not use in production.

Mode 4 — `ViewEncapsulation.None`

Disables all encapsulation. Styles behave as **global CSS**.

```
encapsulation: ViewEncapsulation.None
```

Result:
- Every style in the component's `.scss` file becomes a global style
- These styles apply to the entire application, not just this component
- Styles persist even after the component is destroyed

When to use:
- Root `AppComponent` when you want to include global styles via `styleUrls`
- Rarely — only when you intentionally want component styles to be global

---

Encapsulation Mode Summary

Mode	Styles Leak Out	Global Styles Flow In	Uses Native Shadow DOM
`Emulated` (default)	No	Yes	No (simulated)
`ShadowDom`	No	No	Yes
`ExperimentalIsolatedShadowDom`	No	No (strictly)	Yes
`None`	Yes (global)	Yes	No

---

2. `:host` and `:host-context`

`:host`

Targets the **component's own host element** — the custom element tag itself (e.g. `<book-header>`).

```
// header.scss
:host {
  display: block;
  margin-top: 10px;
}
```

Without `:host`, you have no way to style the outer wrapper element of your component from within its own stylesheet.

With a condition (functional form):

```
:host(.is-active) {
  background-color: #f5e6e8;
  display: block
}

:host(:hover) {
  opacity: 0.9;
}
```

`:host-context`

Styles the component differently based on a **CSS class present on an ancestor element** — anywhere up the DOM tree.

```
// Apply different styles when component is inside a dark-t
heme container
:host-context(.dark-theme) {
  background-color: #222;
  color: #fff;
  display: block;
}

// Apply styles when inside a sidebar
:host-context(.sidebar) .title {
  font-size: 0.9rem;
}
```

Use case: Theme switching — you add `.dark-theme` to the `body` or a wrapper.

3. Maintainable SCSS

Problem

`src/styles.scss` was one large file — 120+ lines mixing body styles, link styles, button styles, and form styles together. Hard to find things, easy to create duplicates (`.cta` was defined twice).

What Was Done

Created `src/scss/` directory with 4 focused partials:

File	Responsibility
`src/scss/_base.scss`	`body`, `ul` — foundational resets
`src/scss/_typography.scss`	All `a` link styles, `.cta` — text/color patterns
`src/scss/_buttons.scss`	`button`, `a.button` — interactive elements
`src/scss/_forms.scss`	`input` — form elements

`src/styles.scss` is now just 4 import lines:

```
@use 'base';
@use 'typography';
@use 'buttons';
@use 'forms';
```

Added `stylePreprocessorOptions` to `angular.json`:

```
"stylePreprocessorOptions": {
  "includePaths": ["src/scss"]
}
```

This tells Angular where to look for Sass files, so any component can import a partial by name alone — no relative paths needed.

Before (in a component):

```
@use '../../../scss/buttons';
```

After:

```
@use 'buttons';
```

---

2. Sass Partials and `@use`

What is a Partial?

A Sass partial is a file prefixed with an underscore (`_buttons.scss`). The underscore signals: *"this file is a piece, not a standalone stylesheet — don't compile me on my own."*

When the **Sass CLI** compiles a directory, it skips `_` files and only compiles top-level files. The partial only outputs CSS when it is imported into another file.

> **Note:** In Angular, the `_` has no functional effect because Angular's build system only compiles files it explicitly knows about (from `styles` array and `styleUrls`). It is a **convention** — a signal to developers.

What is `@use`?

`@use` is the modern Sass rule for importing another file. It replaced the old `@import`.

```
@use 'buttons';       // imports src/scss/_buttons.scss
@use 'typography';    // imports src/scss/_typography.scss
```

4. Interview Questions

Q1. What is Angular View Encapsulation and why does it matter?
View Encapsulation controls whether a component's CSS is scoped to that component or leaks globally. It matters because without it, styles from one component can accidentally break the appearance of other components, especially as an app grows.

Q2. What is the difference between `Emulated` and `ShadowDom` encapsulation?
`Emulated` (default) fakes encapsulation by adding unique HTML attributes to elements and transforming CSS selectors to match — it still allows global styles to flow in. `ShadowDom` uses the browser's native Shadow DOM API for true isolation — global styles cannot penetrate it

Q3. What does `ViewEncapsulation.None` do and when would you use it?
It disables all encapsulation — every style in the component becomes a global style. You'd use it on the root `AppComponent` when you want to define or include global styles using `styleUrls`, or in rare cases where a component intentionally needs to set styles that affect the whole page.

Q4. What is `:host-context()` and what is its current status? `:host-context(selector)` styles a component differently when one of its ancestors matches the given

selector. It is commonly used for theme switching (e.g. `:host-context(.dark-theme)`).

Q5. What is a Sass partial and what is the `_` naming convention?
A Sass partial is a `.scss` file meant to be imported into another file, not compiled on its own. Prefixing the filename with `_` (e.g. `_buttons.scss`) signals this intent. When using the Sass CLI to compile a directory, files starting with `_` are skipped.

Q6. What is `stylePreprocessorOptions.includePaths` in Angular?
It is a configuration in `angular.json` that tells the Sass compiler where to look for files when resolving `@use` and `@import` statements. Once configured, you can import global Sass partials using just the filename from any component, instead of writing long relative paths.

References:

- https://angular.dev/style-guide#choosing-component-selectors
- https://angular.dev/api/core/ViewEncapsulation